SJF (Non-Preemptive) Execution Logic

১. Burst Time (BT) তুলনা করা

SJF-এর প্রধান নিয়ম হলো যে প্রসেসটির **Burst Time (BT)** সবচেয়ে কম, সেটি আগে এক্সিকিউট হবে। তবে শর্ত থাকে যে প্রসেসটিকে অবশ্যই মেমরিতে উপস্থিত (Arrived) থাকতে হবে।

২. যখন BT সমান হয় (Tie-breaker)

যদি একাধিক প্রসেসের BT একই হয়, তবে যার **Arrival Time (AT)** কম তাকে আগে সুযোগ দেওয়া হয়। যদি AT-ও একই হয়, তবে প্রসেস আইডি বা ইনপুট অর্ডার দেখা হয়।

৩. সাধারণ নিয়মাবলী

- এটি একটি **Non-Preemptive** অ্যালগরিদম, অর্থাৎ একবার প্রসেস শুরু হলে শেষ না হওয়া পর্যন্ত CPU অন্য কাউকে দেওয়া হয় না।
- সূত্রসমূহ:
  - CT = ST + BT
  - TAT = CT - AT
  - WT = TAT - BT
  - RT = ST – AT
  - Non-preemptive এর ক্ষেত্রে WT এবং RT সমান হয়।

SJF গাণিতিক উদাহরণ (Solved Table)

আপনার দেওয়া ইনপুটগুলো ব্যবহার করে SJF অনুযায়ী নিচের টেবিলটি তৈরি করা হয়েছে:

QUES SJF:

Process	B.T	A.T	C.T	T.A.T	W.T	R.T
P1	5	4				
P2	2	0				
P3	5	1				
P4	7	6				
P5	3	2				

Process	BT	AT	CT	TAT	WT	RT
P1	5	4	15	11	6	6
P2	2	0	2	2	0	0
P3	5	1	10	9	4	4
P4	7	6	22	16	9	9
P5	3	2	5	3	0	0

Gantt Chart:

P2	P5	P3	P1	P4

0 2 5 10 15 22

SJF C Code

```c
#include <stdio.h>
int main()
{
    int n, i, cnt=0, time=0, idx;
    int pid[20], bt[20], at[20], wt[20], tat[20], ct[20];
    int visited[20] = {0};

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter Process IDs: ");
    for (i=0; i<n; i++)
        scanf("%d", &pid[i]);

    printf("Enter Burst Times: ");
    for (i=0; i<n; i++)
        scanf("%d", &bt[i]);

    printf("Enter Arrival Times: ");
    for (i=0; i<n; i++)
        scanf("%d", &at[i]);

    while (cnt < n)
    {
        idx = -1;
        // Find process with shortest BT among arrived ones
        for (i=0; i<n; i++)
        {
            if (visited[i] == 0 && at[i] <= time)
            {
                if (idx == -1 || bt[i] < bt[idx] || (bt[i] == bt[idx] &&
at[i] < at[idx]))
                {
                    idx = i;
                }
            }
        }
        // If no process has arrived → CPU idle
        if (idx == -1)
        {
            time++;
            continue;
        }

        // Handle idle gap (important fix)
        if (time < at[idx])
            time = at[idx];

        // Execute process
        time += bt[idx];
        ct[idx] = time;
```

```
        tat[idx] = ct[idx] - at[idx];
        wt[idx] = tat[idx] - bt[idx];

        visited[idx] = 1;
        cnt++;
    }
    float avgWT=0, avgTAT=0;

    printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");
    for (i=0; i<n; i++)
    {
        printf("%d\t%d\t%d\t%d\t%d\t%d\n",pid[i], at[i], bt[i], ct[i],
tat[i], wt[i]);

        avgWT += wt[i];
        avgTAT += tat[i];
    }

    printf("\nAverage Waiting Time: %.2f", avgWT/n);
    printf("\nAverage Turnaround Time: %.2f\n", avgTAT/n);

    return 0;
}
```